САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчёт

Домашняя работа № 6
Автоматическое развёртывание приложения с помощью GitHub Actions

Выполнил:

Ермаков Максим Олегович

Группа K3340

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Настроить автоматическое развёртывание бэкенд приложения на удалённом сервере при обновлении кода в определённой ветке репозитория с использованием GitHub Actions или GitLab CI.

Ход работы

1 Проверка исходного состояния

Для работы я использовал приложение бронирования столиков из ЛР4 и сервер 31.77.133.132. Перед настройкой автодеплоя проверил пользователя, адрес сервера, контейнеры и Nginx:

```
whoami
hostname -I
docker compose ps
systemctl is-active nginx
```

```
root@vm329127:/opt/lab4# whoami
root
root@vm329127:/opt/lab4# hostname -I
31.77.133.132 172.17.0.1 172.18.0.1
root@vm329127:/opt/lab4# docker compose ps
NAME                        IMAGE                       COMMAND                  SERVICE              CREATED      STATUS                PORTS
lab4-api-gateway-1          lab4-api-gateway            "docker-entrypoint.s…"   api-gateway          3 days ago   Up 3 days             127.0
.0.1:8200->8200/tcp
lab4-catalog-service-1      lab4-catalog-service        "docker-entrypoint.s…"   catalog-service      3 days ago   Up 3 days
lab4-db-1                   postgres:16                 "docker-entrypoint.s…"   db                   3 days ago   Up 3 days (healthy)   5432/
tcp
lab4-identity-service-1     lab4-identity-service       "docker-entrypoint.s…"   identity-service     3 days ago   Up 3 days
lab4-menu-service-1         lab4-menu-service           "docker-entrypoint.s…"   menu-service         3 days ago   Up 3 days
lab4-rabbitmq-1             rabbitmq:3-management       "docker-entrypoint.s…"   rabbitmq             3 days ago   Up 3 days (healthy)   4369/
tcp, 5671-5672/tcp, 15671-15672/tcp, 15691-15692/tcp, 25672/tcp
lab4-reservation-service-1  lab4-reservation-service   "docker-entrypoint.s…"   reservation-service  3 days ago   Up 3 days
lab4-review-service-1       lab4-review-service         "docker-entrypoint.s…"   review-service       3 days ago   Up 3 days
root@vm329127:/opt/lab4# systemctl is-active nginx
active
root@vm329127:/opt/lab4#
```

Рисунок 1. Исходное состояние сервера и работающие контейнеры

Затем проверил доступ к приложению через публичный IP:

```
curl -i http://31.77.133.132/health
```

```
maksim@MacBook-Air-Maksim ~ % curl -i http://31.77.133.132/health
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Thu, 22 Oct 2026 20:08:19 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 39
Connection: keep-alive
X-Powered-By: Express
Access-Control-Allow-Origin: *
x-request-id: 04e5bba2-7356-48ad-83ae-559b3f39d7b7
ETag: W/"27-OEeR3MgSKG+WBNBm2ZN8Bz3Z0Ag"

{"status":"ok","service":"api-gateway"}%
maksim@MacBook-Air-Maksim ~ %
```

Рисунок 2. Ответ HTTP 200 до добавления метки версии

2 Подготовка ветки и SSH ключа

Я проверил исходную ветку lab4 и создал от неё ветку hw6. Команда git remote -v подтвердила, что origin указывает на мой форк Cheroit/ITMO-ACS-Backend-2026-A.

```
git branch
git log -1 --oneline
git switch -c hw6
git branch --show-current
git remote -v
```

```
maksim@MacBook-Air-Maksim ITMO-ACS-Backend-2026-A % git branch
  hw1,2
  hw4
* lab4
  lr1,-hw3
  lr2,hw4
  lr3,hw5
  main
maksim@MacBook-Air-Maksim ITMO-ACS-Backend-2026-A % git log -1 --oneline
84fa205 (HEAD -> lab4) lab4 and report done
maksim@MacBook-Air-Maksim ITMO-ACS-Backend-2026-A % git switch -c hw6
Switched to a new branch 'hw6'
maksim@MacBook-Air-Maksim ITMO-ACS-Backend-2026-A % git branch --show-current
hw6
maksim@MacBook-Air-Maksim ITMO-ACS-Backend-2026-A % git remote -v
origin  https://github.com/Cheroit/ITMO-ACS-Backend-2026-A.git (fetch)
origin  https://github.com/Cheroit/ITMO-ACS-Backend-2026-A.git (push)
upstream        https://github.com/kantegory/ITMO-ACS-Backend-2026-A.git (fetch)
upstream        https://github.com/kantegory/ITMO-ACS-Backend-2026-A.git (push)
maksim@MacBook-Air-Maksim ITMO-ACS-Backend-2026-A %
```

Рисунок 3. Создание ветки hw6 от готовой ЛР4

Для GitHub Actions я создал отдельную пару ключей ED25519. Публичный ключ нужен серверу для разрешения входа, приватный — автоматизации для подключения.

```
ssh-keygen -t ed25519 -C "github-actions-hw6" \
  -f ~/.ssh/itmo_hw6_deploy
```

```
maksim@MacBook-Air-Maksim ITMO-ACS-Backend-2026-A % ssh-keygen -t ed25519 -C "github-actions-hw6" -f ~/.ssh/itmo_hw6_deploy
Generating public/private ed25519 key pair.
Enter passphrase for "/Users/maksim/.ssh/itmo_hw6_deploy" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /Users/maksim/.ssh/itmo_hw6_deploy
Your public key has been saved in /Users/maksim/.ssh/itmo_hw6_deploy.pub
The key fingerprint is:
SHA256:7WZ++SHTI2ZqQnJoWQaFR16ierCXyvbsZg/AsGO2R18 github-actions-hw6
The key's randomart image is:
+--[ED25519 256]--+
|       ++ .      |
|      oo.o       |
|  . . .o.        |
|   + + .o.       |
|  = * +=E .      |
| o = *=.o.   .   |
|  . =.o+  + *.+  |
|   o oo..+.+o+ o |
|     ++..oo. ..  |
+----[SHA256]-----+
maksim@MacBook-Air-Maksim ITMO-ACS-Backend-2026-A % cd ~/.ssh/
maksim@MacBook-Air-Maksim .ssh % ls
agent                   it_garage               itmo_hw6_deploy         known_hosts
config                  it_garage.pub           itmo_hw6_deploy.pub     known_hosts.old
maksim@MacBook-Air-Maksim .ssh %
```

Рисунок 4. Создание отдельного ключа для автоматического развёртывания

3 Подготовка пользователя deploy

На сервере я проверил наличие rsync для передачи файлов и nano для редактирования. Команда apt install подтвердила, что оба пакета уже установлены:

```
apt install -y rsync nano
getent passwd deploy
```

Пользователя deploy ещё не было, поэтому я создал его без пароля, добавил в группу Docker и передал ему владение каталогом приложения. Для существующего .env установил права 600, а для папки .ssh — 700.

```
adduser --disabled-password --gecos "" deploy
usermod -aG docker deploy
chown -R deploy:deploy /opt/lab4
chmod 600 /opt/lab4/.env
install -d -m 700 -o deploy -g deploy /home/deploy/.ssh
```



Рисунок 5. Создание пользователя и настройка доступа к приложению

Публичный ключ я сохранил в authorized_keys через nano. Файл принадлежит пользователю deploy и доступен ему для чтения и записи:

```
nano /home/deploy/.ssh/authorized_keys
chown deploy:deploy /home/deploy/.ssh/authorized_keys
chmod 600 /home/deploy/.ssh/authorized_keys
```

4 Проверка подключения и настройка Secrets

С локального устройства я подключился новым ключом под пользователем deploy:

```
ssh -i ~/.ssh/itmo_hw6_deploy -o IdentitiesOnly=yes \
  deploy@31.77.133.132
```

После входа whoami вернул deploy. В каталоге /opt/lab4 я проверил Compose и контейнеры. Флаг -q позволяет проверить конфигурацию без вывода подставленных секретов:

```
whoami
cd /opt/lab4
docker compose config -q
docker compose ps
```

```
deploy@vm329127:/opt/lab4$ whoami
deploy
deploy@vm329127:/opt/lab4$ cd /opt/lab4
deploy@vm329127:/opt/lab4$ docker compose config -q
deploy@vm329127:/opt/lab4$ docker compose ps
NAME                        IMAGE                       COMMAND                  SERVICE              CREATED      STATUS                  PORTS
lab4-api-gateway-1          lab4-api-gateway            "docker-entrypoint.s…"   api-gateway          3 days ago   Up 3 days               127.0
.0.1:8200->8200/tcp
lab4-catalog-service-1      lab4-catalog-service        "docker-entrypoint.s…"   catalog-service      3 days ago   Up 3 days
lab4-db-1                   postgres:16                 "docker-entrypoint.s…"   db                   3 days ago   Up 3 days (healthy)     5432/
tcp
lab4-identity-service-1     lab4-identity-service       "docker-entrypoint.s…"   identity-service     3 days ago   Up 3 days
lab4-menu-service-1         lab4-menu-service           "docker-entrypoint.s…"   menu-service         3 days ago   Up 3 days
lab4-rabbitmq-1             rabbitmq:3-management       "docker-entrypoint.s…"   rabbitmq             3 days ago   Up 3 days (healthy)     4369/
tcp, 5671-5672/tcp, 15671-15672/tcp, 15691-15692/tcp, 25672/tcp
lab4-reservation-service-1  lab4-reservation-service   "docker-entrypoint.s…"   reservation-service  3 days ago   Up 3 days
lab4-review-service-1       lab4-review-service         "docker-entrypoint.s…"   review-service       3 days ago   Up 3 days
deploy@vm329127:/opt/lab4$
```

Рисунок 6. Успешная работа с Docker Compose под пользователем deploy

В своём форке я добавил два Repository Secret в Settings → Secrets and variables → Actions. HW6_SSH_PRIVATE_KEY содержит отдельный приватный ключ, HW6_SSH_KNOWN_HOSTS — адрес и публичный ключ сервера для проверки SSH соединения.

Приватный ключ я скопировал в форму GitHub без печати в терминале. Публичный ключ сервера получил из файла с расширением .pub:

```
pbcopy < ~/.ssh/itmo_hw6_deploy
cat /etc/ssh/ssh_host_ed25519_key.pub
```

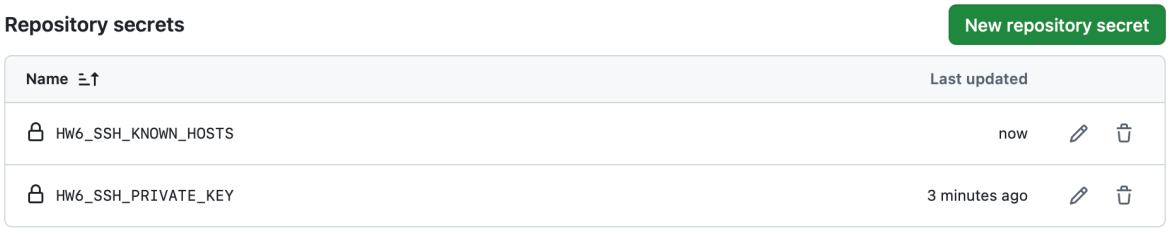


Рисунок 7. Созданные Secrets без отображения их значений

5 Настройка workflow GitHub Actions

В корне репозитория я добавил файл .github/workflows/hw6-deploy.yml. Триггер push ограничен веткой hw6 и изменениями приложения ЛР4 или самого workflow. Для получения кода достаточно права contents: read.



```
.github > workflows > ! hw6-deploy.yml
name: HW6 Deploy backend

on:
  push:
    branches:
      - hw6
    paths:
      - 'БР1.2/Ермаков_Максим/labs/lab4/**'
      - '.github/workflows/hw6-deploy.yml'

permissions:
  contents: read

concurrency:
  group: lab4-server-deploy
  cancel-in-progress: false

jobs:
  deploy:
    if: github.repository == 'Cheroit/ITMO-ACS-Backend-2026-A'
    runs-on: ubuntu-24.04
    timeout-minutes: 40

    steps:
      - name: Get the pushed code
        uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          persist-credentials: false

      - name: Install the transfer tool on the runner
        run: |
          sudo apt-get update
          sudo apt-get install -y --no-install-recommends rsync openssh-client curl

      - name: Configure SSH on the runner
        env:
          SSH_PRIVATE_KEY: ${{ secrets.HW6_SSH_PRIVATE_KEY }}
          SSH_KNOWN_HOSTS: ${{ secrets.HW6_SSH_KNOWN_HOSTS }}
        run: |
          test -n "$SSH_PRIVATE_KEY"
          test -n "$SSH_KNOWN_HOSTS"
```

Рисунок 8. Workflow с триггером на ветку hw6 и подготовкой runner

Задание deploy выполняется на runner Ubuntu 24.04. Runner получает код коммита и подключается к серверу по SSH. Параметр concurrency исключает одновременное выполнение двух деплоев этого workflow, а начатый запуск не отменяется новым push.

6 Передача кода и обновление контейнеров

Для передачи исходников workflow использует rsync. Исходный каталог указан с завершающим слешем, поэтому на сервер передаётся его содержимое. .env исключён из синхронизации и остаётся на сервере:

```
rsync -az --delete \
  --exclude='.env' --exclude='.env.*' \
  --exclude='.git/' --exclude='node_modules/' \
  --exclude='dist/' --exclude='dbs/' --exclude='.DS_Store' \
  --exclude='*.pdf' --exclude='*.docx' --exclude='*.zip' \
  --exclude='screenshots/' --exclude='outputs/' \
  'БР1.2/Ермаков_Максим/labs/lab4/' hw6-server:/opt/lab4/
```

Затем по SSH в каталоге /opt/lab4 последовательно выполняются проверка Compose, сборка образов и обновление контейнеров:

```
docker compose config -q
docker compose build
docker compose up -d --no-build --wait --wait-timeout 180
docker compose ps
```

Сборка использует существующие Dockerfile. Параметр --wait ожидает состояние running/healthy. Затем runner проверяет через Nginx адреса /health и /api/v1/restaurants с помощью curl: HTTP ошибка завершает шаг неуспешно.

7 Изменение кода для проверки автодеплоя

Чтобы проверить получение сервером нового кода, я добавил поле version со значением hw6-v1 в обработчик /health файла lab4/src/common/service-app.ts.

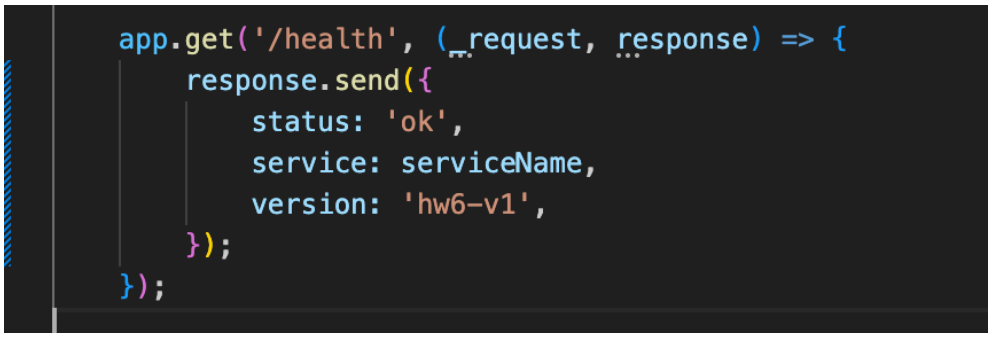


Рисунок 9. Метка hw6-v1 в ответе обработчика проверки состояния

8 Отправка изменений и автоматический запуск

Я добавил workflow и изменённый обработчик в коммит, после чего отправил ветку hw6 в свой репозиторий:

```
git add .github/workflows/hw6-deploy.yml \
  'БР1.2/Ермаков_Максим/labs/lab4/src/common/service-app.ts'
git commit -m "hw6: configure automatic deployment"
git push -u origin hw6
```

```
maksim@MacBook-Air-Maksim ITMO-ACS-Backend-2026-A % git push -u origin hw6
Enumerating objects: 96, done.
Counting objects: 100% (96/96), done.
Delta compression using up to 8 threads
Compressing objects: 100% (88/88), done.
Writing objects: 100% (91/91), 11.80 MiB | 2.34 MiB/s, done.
Total 91 (delta 18), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (18/18), completed with 4 local objects.
remote:
remote: Create a pull request for 'hw6' on GitHub by visiting:
remote:      https://github.com/Cheroit/ITMO-ACS-Backend-2026-A/pull/new/hw6
remote:
To https://github.com/Cheroit/ITMO-ACS-Backend-2026-A.git
 * [new branch]      hw6 -> hw6
branch 'hw6' set up to track 'origin/hw6'.
maksim@MacBook-Air-Maksim ITMO-ACS-Backend-2026-A %
```

Рисунок 10. Успешная отправка ветки hw6 в origin

После push во вкладке Actions автоматически появился запуск HW6 Deploy backend для коммита d19c26e. Сборку и обновление контейнеров выполнял workflow на сервере.

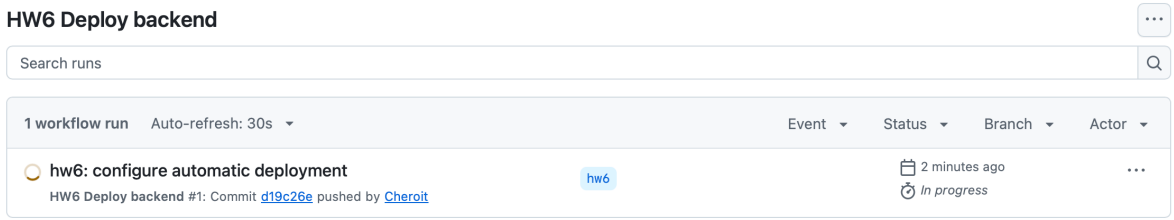


Рисунок 11. Запуск workflow после push коммита d19c26e

Ключ -u в команде push связал локальную ветку с origin/hw6. Для следующих обновлений этой ветки достаточно отправить новый коммит командой git push.

9 Автоматическая сборка и результат запуска

В журнале Actions я проверил выполнение шагов и запуск docker compose build через SSH. После сборки workflow обновил контейнеры и выполнил HTTP проверки.

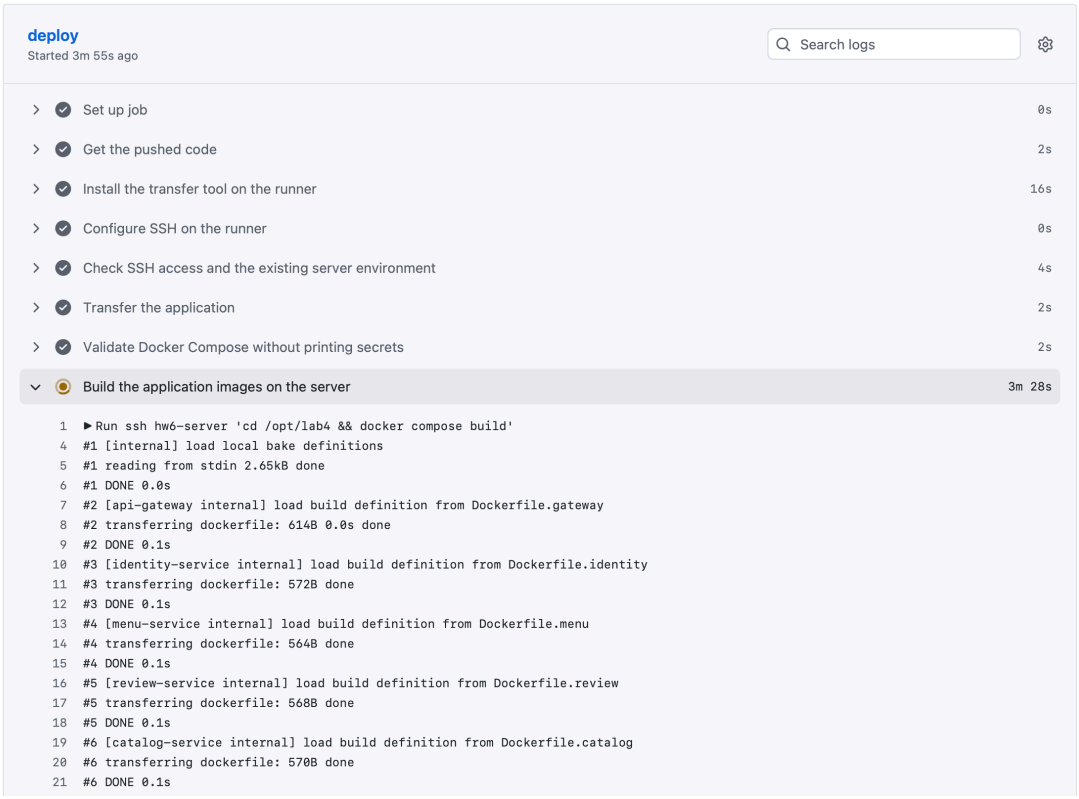


Рисунок 12. Выполнение шага сборки Docker образов на сервере

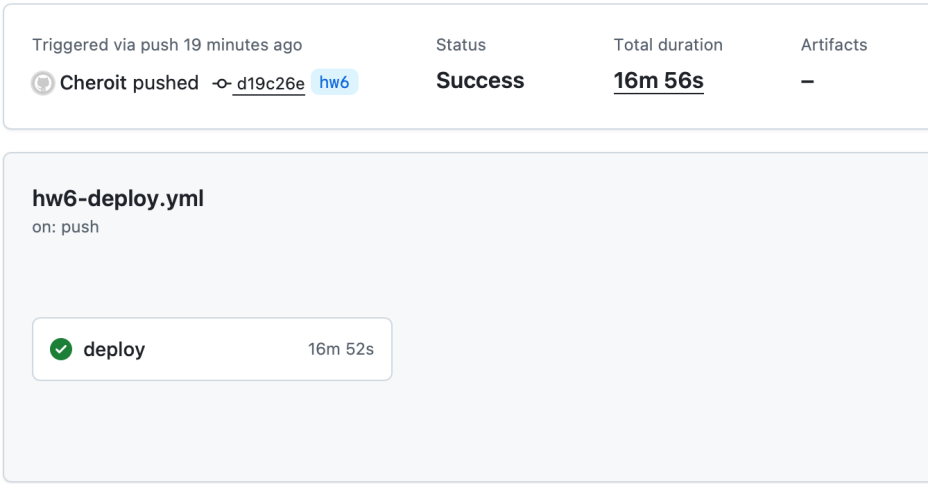


Рисунок 13. Успешный запуск workflow за 16 минут 56 секунд

10 Проверка приложения после автодеплоя

После завершения Actions я повторил запрос с MacBook к публичному адресу:

```
curl -i http://31.77.133.132/health
```

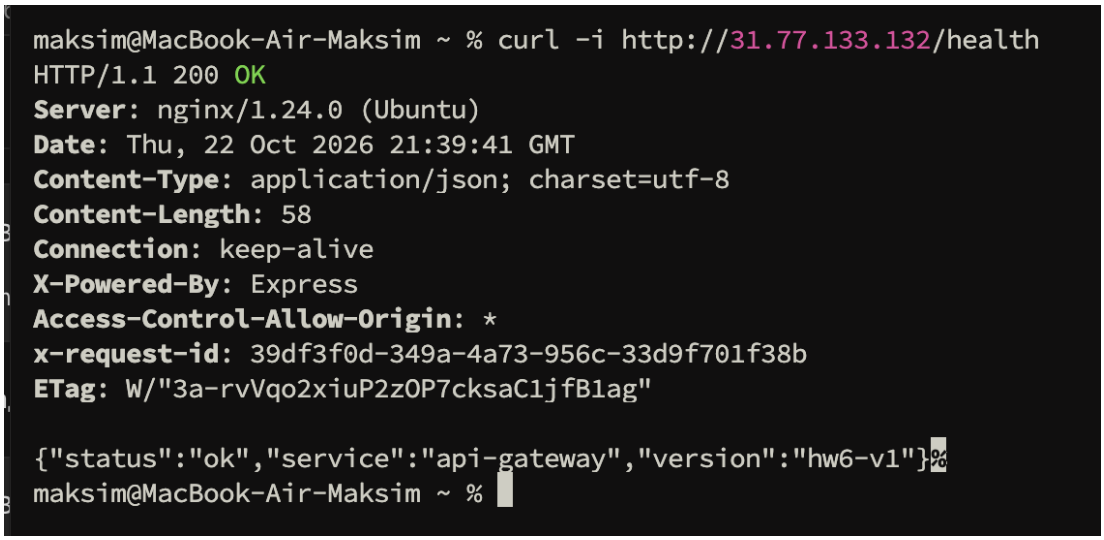


Рисунок 14. Ответ HTTP 200 с новой меткой hw6-v1

В ответе появилось поле version: hw6-v1, которого не было до настройки. Это подтвердило, что через Nginx доступен обновлённый код. На сервере я также проверил контейнеры и Nginx:

```
docker compose ps
systemctl is-active nginx
```

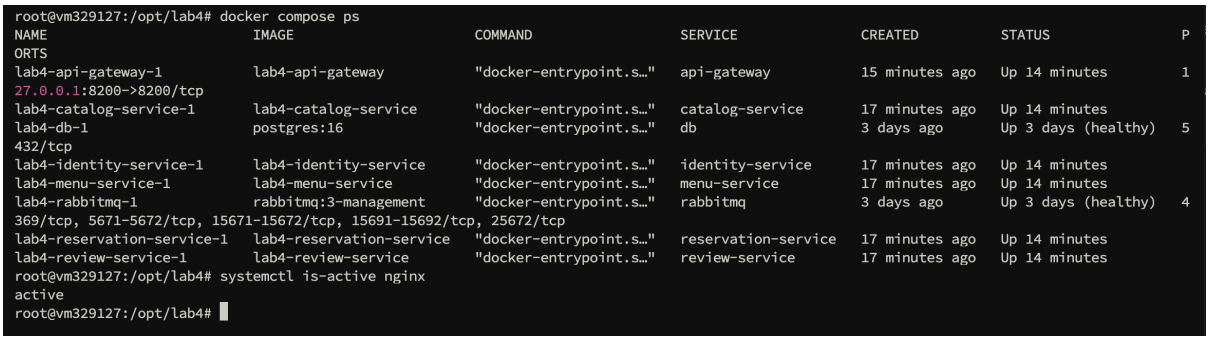


Рисунок 15. Работающие контейнеры и активный Nginx после обновления

Вывод

Я настроил автоматическое развёртывание приложения через GitHub Actions при push в ветку hw6. Workflow передаёт исходники, собирает образы, обновляет контейнеры и проверяет публичный API. Успешный запуск коммита d19c26e и появление метки hw6-v1 подтвердили работу автодеплоя. Для дальнейшего обновления приложения достаточно отправить изменения в настроенную ветку.